Open in app ↗ Sign up Sign in

Medium Search Write

Computational Law + Rules As Code + Docassemble + Legaltech +

Introducing L4-Docassemble

SMU CClaw brings logic-based legal encodings, defaults and exceptions, reusable rules and data structures, explanations, rapid prototyping, and deeper legal reasoning to open source legal expert systems.

Jason Morris 9 min read · May 5, 2021

-- 1 --

At SMU's Centre for Computational Law we are working on an open source domain specific language for law, called L4. The idea is that you should be able to write legal rules, like laws or contracts, in L4, and other applications should be able to translate that encoding into other forms to do useful things.

One of the useful things that we want people to be able to do with L4 is to build expert systems. An expert system is an application that is given a question to answer, collects relevant information from a user, and then provides the answer an expert would give. A familiar example of an expert system is TurboTax, which answers the question of "how should I fill in my tax forms?"

Open Source Legal Expert System Tools: What are they missing?

If you are interested in developing legal expert systems, and you want to do it using open source software (and you should), your best option right now is Docassemble. Docassemble is a popular open source interview tool that figures out what question to ask, and in what order, and allows you to easily customize the display of questions. You can also make it do anything else you want if you are proficient with the Python programming language.

While it is the best open source option available, and has much to recommend it, Docassemble does not have some of the features of the best commercial and closed-source tools for legal expert system development, like Oracle Intelligent Advisor, Neota Logic, and DataLex. Here are some of the things that currently there is no easy way to do when developing legal expert systems using open source tools.[1]

Logical Representations of Laws and Contracts

OIA, Neota Logic, and DataLex allow you to encode what you know about the law in a declarative, logic-based language, which is *extremely helpful* when encoding laws in a way that is intended to be reusable, such as in Rules as Code drafting process.

Prototype User Interfaces

In OIA, Neota Logic, and DataLex, merely by specifying a data structure, a set of rules, and a question to answer, you can get a working prototype expert system interface built for you automatically.

Explanations for Answers

OIA, Neota Logic, and DataLex all include natural language explanations for the answers their expert systems generate.

Legal Expert System Tools: What are they missing?

There are also a number of capabilities that would be helpful when building legal expert system tools, but currently do not exist anywhere. Not even OIA, Neota Logic, and DataLex can do these things.

Defeasibility

Laws and contracts are frequently written with rules that override other rules. It would simplify the task of encoding these rules, and maintaining the encodings when the rules change, if it was possible to encode those defeating relationships as they appear in the law, instead of having to combine defeating rules with the rules they defeat.

Modular Rules and Data Structures

In all the existing tools, whatever data structure you use for designing the interview interface is also the data structure that you use for encoding the rules. But the best way to collect information depends on the problem you are trying to solve, not the law, and one law can solve many problems, each of which might benefit from collecting the relevant information in a different way. Encodings of rules would be more re-usable if information about how to collect information was taken out of them, so that you could use the same rules with multiple different interfaces without having to rewrite the rules.

So we set about to see if it would be possible to make it easier to use a declarative logic encoding to power a Docassemble legal expert system.

Multiple Justifications

Existing tools like OIA, Neota Logic, and DataLex are able to give explanations for their answers. But the explanation that they provide is a description of how that tool derived the answer. These tools are not capable

of telling the user whether the same conclusion might have been reached in a different way on the basis of the same facts and rules.

This has practical benefits for legal expert systems where you want to allow the user to choose the most compelling explanation to put in a document. But more immediately, being able to see all of the ways that your encoding of the rules can reach the same conclusion gives the legal knowledge engineer a depth of insight into how their encoding works that is not otherwise possible. And in a Rules as Code drafting process, that insight may allow legislative and contract drafters to see problems with the actual rules that they would otherwise have missed.

Most obviously, being able to see all the multiple justifications that can be generated for a conclusion allows you to see when there are sections of your encoding (or your law or contract), that are redundant.

Hypothetical Reasoning

Often, legal questions are asked with imperfect information. Most sophisticated expert systems are able to distinguish between something that is known to be false, and something that is just not known. If a necessary relevant fact is unknown, then they report the conclusion as unknown, instead of false.

It would be more helpful if a legal expert system tool could say “These are the combinations of unknown facts that would need to be true in order for the conclusion to be true.” That sort of hypothetical reasoning can be used to give contingent answers to legal questions, and to generate plans to achieve a user’s objectives. It can also be used to do analysis of rules and encodings without having to provide facts.

What is L4-Docassemble?

L4-Docasemble is our attempt to fill in some of those missing pieces in the open source legal expert tool landscape.

We are currently experimenting with using s(CASP) as a language for powering expert systems, and others on the team are actively working on translating L4 code into s(CASP) for that purpose. I needed to give Docassemble the ability to use those translated s(CASP) encodings. We did that with the docassemble-scasp package, available on GitHub.

We also wanted to be able to automatically generate prototype interfaces on the basis of a data structure. We achieved that by using the docassemble-datatypes package that we introduced at DocaCon 2020, and available on GitHub.

Then we needed the ability to generate an interview on the basis of an s(CASP) encoding and a data structure. To achieve that we created a data structure format called LExSIS (Legal EXpert System Interface Schema), and implemented LExSIS for use with s(CASP) encodings in the docassemble-l4 package, which is available on GitHub.[2]

Then we put all of them together in a single docker container that pre-installs the above packages, s(CASP), and docassemble. That is L4-Docassemble, and it is available at GitHub now. Note that the L4-Docassemble package doesn't contain any code, it is just a convenient way of installing everything at once.

The result is a legal expert system tool with the following features:

- Logical Representation of Rules *

- Rapid Prototyping *
- Natural Language Explanations *
- Defeasibility +
- Modular Data Structures and Rules +
- Multiple Justifications +

As mentioned, the features marked "*" do not exist in any other open source tool of which we are aware, and the features marked "+" are completely novel for expert system development tools.

Get Jason Morris's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

We are continuing to work on adding hypothetical reasoning, which we expect to be able to add in a couple of months.

How do you Use L4-Docassemble?

1. Install it.
2. Upload your s(CASP) and LExSIS files to the static folder in the docassemble playground.
3. Run the interview generation tool, which creates an interview and installs it onto your docassemble server for you.
4. Modify the interview however you like.

All of the interview parts that are generated by L4-Docassemble are just defaults that can be replaced by the interview developer by using “plain old docassemble.”

What Does Using L4-Docassemble Look Like?

For a quick 3-minute introduction, check out this video:

As a demonstration of L4-Docassemble we created an expert system capable of answering questions about a piece of legislation called Rule 34. The first step was to encode Rule 34 in s(CASP). This is an excerpt of that encoding:

```
% RULE 34(5)
% (5)  Despite paragraph (1)(b), but subject to paragraph (1)(a) and
(c) to (f),
% a locum solicitor may accept an executive appointment in a business
entity which
% does not provide any legal services or law-related services, if all
```

```
of the
% conditions set out in the Second Schedule are satisfied.

according_to(r34_5,may(LP,accept,EA)) :-
    legal_practitioner(LP),
    locum_solicitor(LP),
    executive_appointment_in_a_business_entity(EA,BE),
    not provides_legal_or_law_related_services(BE),
    conditions_of_second_schedule_satisfied.

opposes(r34_1,must_not(LP,accept,EA),r34_5,may(LP,accept,EA)).
opposes(r34_5,may(LP,accept,EA),r34_1_b,must_not(LP,accept,EA)).

overrides(r34_1,must_not(LP,accept,EA),r34_5,may(LP,accept,EA)).
overrides(r34_5,may(LP,accept,EA),r34_1_b,must_not(LP,accept,EA)).

provides_legal_or_law_related_services(BE) :-
    provides(BE,Serv),
    legal_service(Serv).

provides_legal_or_law_related_services(BE) :-
    provides(BE,Serv),
    law_related_service(Serv).
```

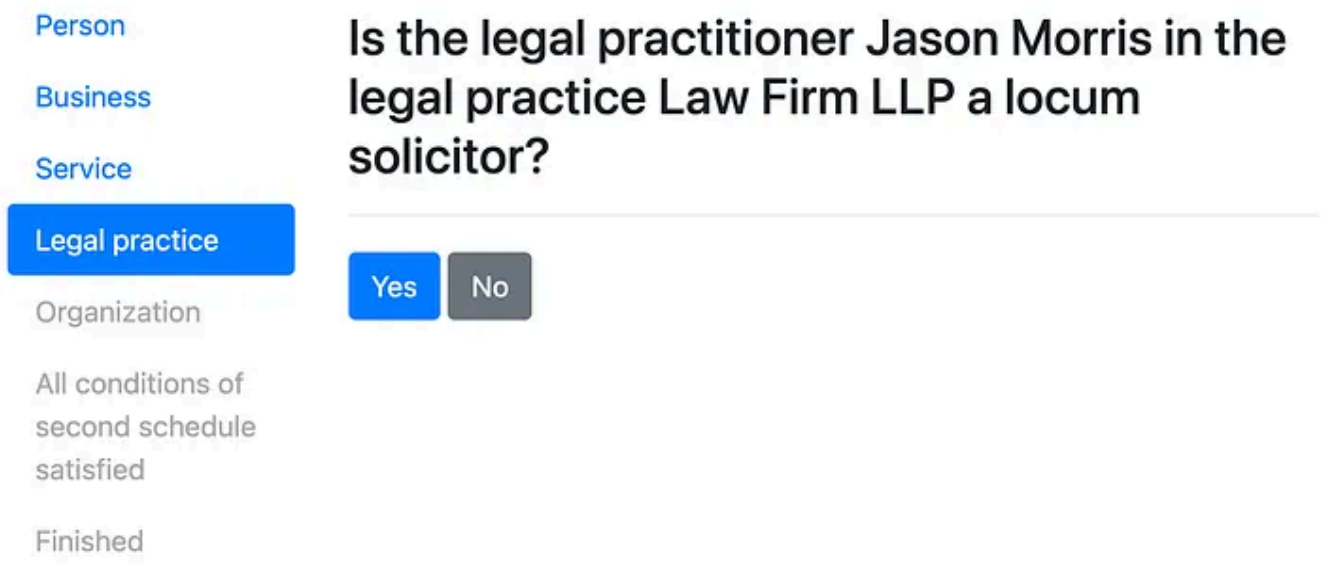
Note how the “despite” and “subject to” parts of the original law were translated into “opposes” and “overrides” statements in the code, without having to add anything to the other rules. That is what it means to be able to write rules with defeasibility.

The next step is to write a LExSIS file. Rule 34 is a complicated piece of law with almost 100 input factors, so our demonstration LExSIS file is quite long. Here is an excerpt that describes law practices:

```
rules: r34.pl
query: legally_holds(Rule,must_not(Lawyer,accept,Position))
data:
...
  - name: legal_practice
    type: String
    any: Are there any legal practices?
    another: Is there another legal practice?
    ask: What is the name of the legal practice?
    tell: the legal practice {X}
```

```
    minimum: 0
    encodings:
      - law_practice(X)
    attributes:
    - name: joint_law_venture
      ask: Is {Y} a joint law venture?
      type: Boolean
      encodings:
        - joint_law_venture(Y)
    - name: formal_law_alliance
      type: Boolean
      ask: Is {Y} a formal law alliance?
      encodings:
        - formal_law_alliance(Y)
    - name: foreign_law_practice
      type: Boolean
      ask: Is {Y} a foreign law practice?
      encodings:
        - foreign_law_practice(Y)
    - name: jurisdiction
      type: Enum
      ask: What is the jurisdiction of {Y}?
      options:
        singapore: Singapore
        other: Somewhere Else
      encodings:
        - jurisdiction(Y,X)
    - name: legal_practitioner
      type: Object
      source: person
      minimum: 0
      any: Are there any legal practitioners in {Y}?
      another: Is there another legal practitioner in {Y}?
      ask: Who is a legal practitioner in {Y}?
      tell: the legal practitioner {X} in {Y}
      encodings:
        - legal_practitioner(X)
        - in(X,Y)
      attributes:
        - name: locum_solicitor
          ask: Is {Y} a locum solicitor?
          type: Boolean
          encodings:
            - locum_solicitor(Y)
        - name: primary_occupation
          type: Enum
          ask: What is the primary occupation of {Y}?
          encodings:
            - primary_occupation_of(Y,X)
          options:
            practicing_as_a_lawyer: Practicing as a lawyer
            something_else: Something Else
```

You give those two files to the interview generation tool, and it creates the interview, installs it, and gives you a link. You click on the link, the interview asks all the relevant questions, one at a time, and then displays the answers using the instructions in the LExSIS files and the answers the user has already provided. Here is what one of the questions generated by the LExSIS code above looks like:



A question screen in an L4-Docassemble interview.

Here's what the final screen of the demonstration interview will look like.

An example of an answer from an L4-Docassemble interview

It answers the question, explains the answers in natural language, and it provides all the possible justifications for each answer. And features of

LExSIS allow you to provide the source text of the encoded rules, and links to the source material, as meta-data that can be displayed to the user on request.

Legal Expert Systems in One Line of Code

Imagine a situation in which someone has already written an s(CASP) encoding of, and a LExSIS file for, the law you want to ask a question about.

How do you use L4-Docassemble to create a legal expert system that answers a completely different question related to the same law?

1. Change the `query` entry in your LExSIS file.
2. Rerun the interview generator.
3. There is no step 3.

You can try this yourself by changing the query in the demo LExSIS file to `query: business_entity(Organization)`, and re-running the interview generator.

> Using L4-Docassemble, building a new legal expert system to answer a different question about the same rules requires a single line of code.

This is the huge promise of the rules as code approach to legal expert system development. What you get from the investment of encoding your rules is not an app that can answer *one* question. You get an app factory, that can instantaneously generate an app that will answer *any* question you can pose.

Thanks

A huge thank you to **Joaquín Arias Herrero**, of the Universidad Rey Juan Carlos, who is one of the authors of s(CASP), both for creating the tool and for being a great resource to me in learning how to use it properly. Thanks to **Jonathan Pyle** for creating Docassemble and for his responsiveness and support to all its users. Thanks also to my colleagues at SMU CCLaw, in particular our Principal Investigator **Meng Weng Wong**, who has allowed me to spend time scratching my own itches. I would also like to specifically thank my colleagues **Alfred Ang**, **Regina Cheong**, and **Liyana Mohd Ramthan** who have slogged through the idempotentcy wilderness with me over the last few months.

Notes:

1. There are open source reasoning tools like Blawx and closed source tools like Regulation as a Platform that feature logical representations and defeasibility, but that do not have features for the development of user interfaces. Such tools are excluded from the analysis.
2. I'll write more about LExSIS in a later blog post.

Computational Law + Rules As Code + Docassemble + Legaltech +

Written by Jason Morris

169 followers · 20 following

Rules as Code Director at Service Canada, CEO Lexpedite.ca. Previous: Lawyer; ABA Innovation Fellow; Sessional Instructor; Computational Law Researcher

Published in Rules as Code Diary

27 followers · Last published Jan 10, 2022

A sporadic diary by Jason Morris of Lexpedite Legal Technology Ltd.